Experiment : 6

Aim: Implement Stack ADT and perform arithmetic expression evaluation.

Stack ADT (Abstract Data Type)

Definition:

A **stack** is a linear data structure that follows **LIFO (Last In First Out)** order: the last element inserted is the first to be removed.

Basic Operations of Stack:

1. **push(element):** Add an element to the top.
2. **pop():** Remove and return the top element.
3. **peek() / top():** Return the top element without removing it.
4. **isEmpty():** Check if the stack is empty.

Stacks are commonly used for:

- Expression evaluation (postfix/prefix)
- Undo/redo operations
- Function call management in recursion

2. Algorithm for Arithmetic Expression Evaluation (Postfix)

Assumptions: The expression is in **postfix notation** (Reverse Polish Notation).

Algorithm:

1. Initialize an empty stack.
2. Read the expression from left to right.
3. For each token:
    - If the token is an **operand** (number), push it onto the stack.
    - If the token is an **operator** (+, -, *, /):
        1. Pop the top two elements from the stack.

Dept of AIML

2. Apply the operator: result = operand1 operator operand2

3. Push the result back onto the stack.

4. After processing all tokens, the stack will have one element: the result.

Program:

```
class Stack:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("Stack underflow")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            return None

# Function to evaluate postfix expression
def evaluate_postfix(expression):
    stack = Stack()
    operators = '+-*/'

    for token in expression.split():
        if token.isdigit():
            stack.push(int(token))
        elif token in operators:
            operand2 = stack.pop()
            operand1 = stack.pop()
            if token == '+':
                stack.push(operand1 + operand2)
            elif token == '-':
                stack.push(operand1 - operand2)
            elif token == '*':
                stack.push(operand1 * operand2)
```

Dept of AIML

```
        elif token == '/':
            stack.push(int(operand1 / operand2))  # integer division
        else:
            raise ValueError(f"Invalid token: {token}")

    return stack.pop()

# Example usage
postfix_expr = "5 3 + 8 2 - *"  # Equivalent to (5+3)*(8-2)
result = evaluate_postfix(postfix_expr)
print(f"Postfix Expression: {postfix_expr}")
print(f"Evaluated Result: {result}")
```

OUTPUT:

Postfix Expression: 5 3 + 8 2 - *
Evaluated Result: 48

Dept of AIML